

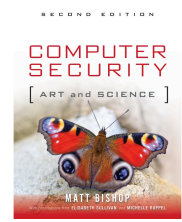
Security Policies

Chapter 4



Overview

- Overview
- Policies
- Trust
- Nature of Security Mechanisms
- Policy Expression Languages
- Limits on Secure and Precise Mechanisms



Security Policy

- Policy partitions system states into:
  - Authorized (secure)
    - These are states the system can enter
  - Unauthorized (nonsecure)
    - If the system enters any of these states, it's a security violation
- Secure system
  - Starts in authorized state
  - Never enters unauthorized state



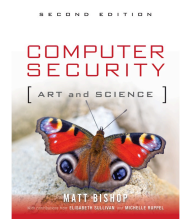
Confidentiality

- *X* set of entities, *I* information
- *I* has the *confidentiality* property with respect to *X* if no $x \in X$ can obtain information from *I*
- *I* can be disclosed to others
- Example:
  - *X* set of students
  - *I* final exam answer key
  - *I* is confidential with respect to *X* if students cannot obtain final exam answer key



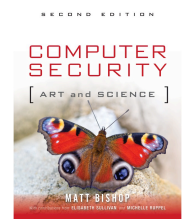
Integrity

- *X* set of entities, *I* information
- *I* has the *integrity* property with respect to *X* if all $x \in X$ trust information in *I*
- Types of integrity:
  - Trust *I*, its conveyance and protection (data integrity)
  - *I* information about origin of something or an identity (origin integrity, authentication)
  - *I* resource: means resource functions as it should (assurance)



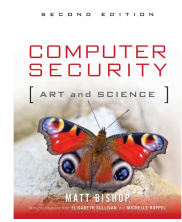
Availability

- *X* set of entities, *I* resource
- *I* has the *availability* property with respect to *X* if all $x \in X$ can access *I*
- Types of availability:
  - Traditional: *x* gets access or not
  - Quality of service: promised a level of access (for example, a specific level of bandwidth); *x* meets it or not, even though some access is achieved



Policy Models

- Abstract description of a policy or class of policies
- Focus on points of interest in policies
  - Security levels in multilevel security models
  - Separation of duty in Clark-Wilson model
  - Conflict of interest in Chinese Wall model



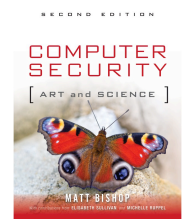
Mechanisms

- Entity or procedure that enforces some part of the security policy
  - Access controls (like bits to prevent someone from reading a homework file)
  - Disallowing people from bringing CDs and floppy disks into a computer facility to control what is placed on systems



Question

- Policy disallows cheating
  - Includes copying homework, with or without permission
- CS class has students do homework on computer
- Anne forgets to read-protect her homework file
- Bill copies it
- Who breached security?
  - Anne, Bill, or both?



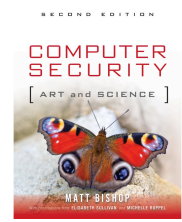
Answer Part 1

- Bill clearly breached security
  - Policy forbids copying homework assignment
  - Bill did it
  - System entered unauthorized state (Bill having a copy of Anne's assignment)
- If not explicit in computer security policy, certainly implicit
  - Not credible that a unit of the university allows something that the university as a whole forbids, unless the unit explicitly says so



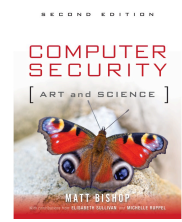
Answer Part #2

- Anne didn't protect her homework
  - Not required by security policy
- She didn't breach security
- If policy said students had to read-protect homework files, then Anne did breach security
  - She didn't do this



Types of Security Policies

- Military (governmental) security policy
  - Policy primarily protecting confidentiality
- Commercial security policy
  - Policy primarily protecting integrity
- Confidentiality policy
  - Policy protecting only confidentiality
- Integrity policy
  - Policy protecting only integrity



Integrity and Transactions

- Begin in consistent state
  - "Consistent" defined by specification
- Perform series of actions (*transaction*)
  - Actions cannot be interrupted
  - If actions complete, system in consistent state
  - If actions do not complete, system reverts to a consistent state



Trust

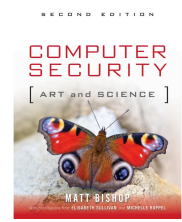
Administrator installs patch

1. Trusts patch came from vendor, not tampered with in transit
2. Trusts vendor tested patch thoroughly
3. Trusts vendor's test environment corresponds to local environment
4. Trusts patch is installed correctly



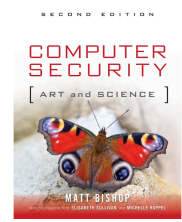
Trust in Formal Verification

- Gives formal mathematical proof that given input *i*, program *P* produces output *o* as specified
- Suppose a security-related program *S* formally verified to work with operating system *O*
- What are the assumptions?



Trust in Formal Methods

1. Proof has no errors
   - Bugs in automated theorem provers
2. Preconditions hold in environment in which *S* is to be used
3. *S* transformed into executable *S′* whose actions follow source code
   - Compiler bugs, linker/loader/library problems
4. Hardware executes *S′* as intended
   - Hardware bugs (Pentium `f00f` bug, for example)



Types of Access Control

- Discretionary Access Control (DAC, IBAC)
  - Individual user sets access control mechanism to allow or deny access to an object
- Mandatory Access Control (MAC)
  - System mechanism controls access to object, and individual cannot alter that access
- Originator Controlled Access Control (ORCON, ORGCON)
  - Originator (creator) of information controls who can access information



Policy Languages

- Express security policies in a precise way
- High-level languages
  - Policy constraints expressed abstractly
- Low-level languages
  - Policy constraints expressed in terms of program options, input, or specific characteristics of entities on system



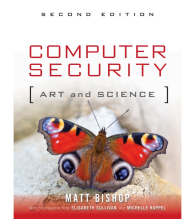
High-Level Policy Languages

- Constraints expressed independent of enforcement mechanism
- Constraints restrict entities, actions
- Constraints expressed unambiguously
  - Requires a precise language, usually a mathematical, logical, or programming-like language



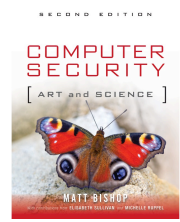
Example: Ponder

- Security and management policy specification language
- Handles many types of policies
  - Authorization policies
  - Delegation policies
  - Information filtering policies
  - Obligation policies
  - Refrain policies



Entities

- Organized into hierarchical domains
- Network administrators
  - *Domain* is /NetAdmins
  - Subdomain for net admin trainees is
  - /NetAdmins/Trainees
- Routers in LAN
  - Domain is /localnet
  - Subdomain that is a testbed for routers is
  - /localnet/testbed/routers



Authorization Policies

- Allowed actions: netadmins can enable, disable, reconfigure, view configuration of routers

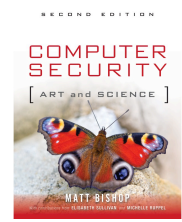
```
inst auth+ switchAdmin {
        subject /NetAdmins;
        target  /localnetwork/routers;
        action  enable(), disable(), reconfig(), dumpconfig();
}
```



Authorization Policies

- Disallowed actions: trainees cannot test performance between 8AM and 5PM

```
inst auth- testOps {
    subject /NetEngineers/trainees;
    target  /localnetwork/routers;
    action  testperformance();
    when    Time.between("0800", "1700");
}
```



Delegation Policies

- Delegated rights: net admins delegate to net engineers the right to enable, disable, reconfigure routers on the router testbed

```
inst deleg+ (switchAdmin) delegSwitchAdmin {
    grantee   /NetEngineers;
    target    /localnetwork/testNetwork/routers;
    action    enable(), disable(), reconfig();
    valid     Time.duration(8);
}
```



Information Filtering Policies

- Control information flow: net admins can dump everything from routers between 8PM and 5AM, and config info anytime

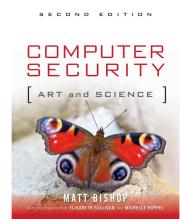
```
inst auth+ switchOpsFilter {
     subject   /NetAdmins;
     target    /localnetwork/routers;
     action    dumpconfig(what)
               { in partial = "config"; }
               if (Time.between("2000", "0500")){
                    in partial = "all"; }
}
```



Refrain Policies

- Like authorization denial policies, but enforced by the *subjects*: net engineers cannot send test results to net developers while testing in progress

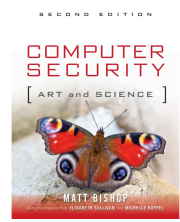
```
inst refrain testSwitchOps {
    subject  s=/NetEngineers;
    target   /NetDevelopers;
    action   sendTestResults();
    when    s.teststate="in progress"
}
```



Obligation Policies

- Must take actions when events occur: on 3rd login failure, net security admins will disable account and log event

```
inst oblig loginFailure {
    on        loginfail(userid, 3);
    subject   s=/NetAdmins/SecAdmins;
    target    t=/NetAdmins/users ^ (userid);
    do        t.disable() -> s.log(userid);
}
```



Example

- Policy: separation of duty requires 2 different members of Accounting approve check

```
inst auth+ separationOfDuty {
    subject   s=/Accountants;
    target    t=checks;
    action    approve(), issue();
    when      s.id <> t.issuerid;
}
```



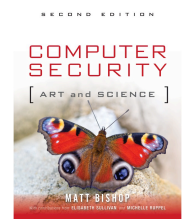
DTEL

- Basis: access can be constrained by types
- Combines elements of low-level, high-level policy languages
  - Implementation-level constructs express constraints in terms of language types
  - Constructs do not express arguments or inputs to specific system commands



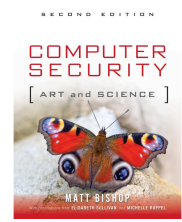
Example

- Goal: users cannot write to system binaries
- Subjects in administrative domain can
  - User must authenticate to enter that domain
- Subjects belong to domains:
  - *d_user* ordinary users
  - *d_admin* administrative users
  - *d_login* for login
  - *d_daemon* system daemons



Types

- Object types:
  - *t_sysbin* executable system files
  - *t_readable* readable files
  - *t_writable* writable files
  - *t_dte* data used by enforcement mechanisms
  - *t_generic* data generated from user processes
- For example, treat these as partitions
  - In practice, files can be readable and writable; ignore this for the example

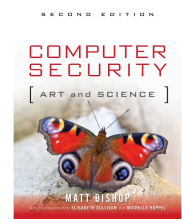


Domain Representation

- Sequence
  - First component is list of programs that start in the domain
  - Other components describe rights subject in domain has over objects of a type

    `(crwd->t_writable)`

    means subject can create, read, write, and list (search) any object of type t_writable



d_daemon Domain

```
domain d_daemon = (/sbin/init),
        (crwd->t_writable),
        (rd->t_generic, t_readable, t_dte),
        (rxd->t_sysbin),
        (auto->d_login);
```

- Compromising subject in *d_daemon* domain does not enable attacker to alter system files
  - Subjects here have no write access
- When /sbin/init invokes login program, login program transitions into *d_login* domain



d_admin Domain

```
domain d_admin =
        (/usr/bin/sh, /usr/bin/csh, /usr/bin/ksh),
        (crwxd->t_generic),
        (crwxd->t_readable, t_writable, t_dte, t_sysbin),
        (sigtstp->d_daemon);
```

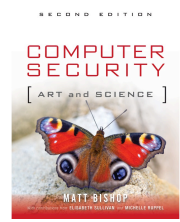
- *sigtstp* allows subjects to suspend processes in *d_daemon* domain
- Admin users use a standard command interpreter



d_user Domain

```
domain d_user =
        (/usr/bin/sh, /usr/bin/csh, /usr/bin/ksh),
        (crwxd->t_generic),
        (rxd->t_sysbin),
        (crwd->t_writable),
        (rd->t_readable, t_dte);
```

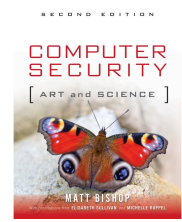
- No auto component as no user commands transition out of it
- Users cannot write to system binaries



d_login Domain

```
domain d_login =
        (/usr/bin/login),
        (crwd->t_writable),
        (rd->t_readable, t_generic, t_dte),
        setauth,
        (exec->d_user, d_admin);
```

- Cannot execute anything except the transition
  - Only /usr/bin/login in this domain
- *setauth* enables subject to change UID
- *exec* access to *d_user*, *d_admin* domains



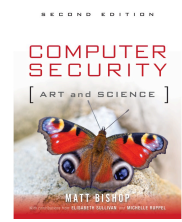
Set Up

```
initial_domain = d_daemon;
```

- System starts in *d_daemon* domain

```
assign –r t_generic /;
assign –r t_writable /usr/var, /dev, /tmp;
assign –r t_readable /etc;
assign –r –s dte_t /dte;
assign –r –s t_sysbin /sbin, /bin,
                      /usr/bin, /usr/sbin;
```

- These assign initial types to objects
- –r recursively assigns type
- –s binds type to name of object (delete it, recreate it, still of given type)



Add Log Type

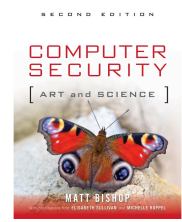
- Goal: users can't modify system logs; only subjects in *d_admin*, new *d_log* domains can

```
type t_readable, t_writable, t_sysbin,
                    t_dte, t_generic, t_log;
```

- New type *t_log*

```
domain d_log =
        (/usr/sbin/syslogd),
        (crwd->t_log),
        (rwd->t_writable),
        (rd->t_generic, t_readable);
```

- New domain *d_log*



Fix Domain and Set-Up

```
domain d_daemon =
        (/sbin/init),
        (crwd->t_writable),
        (rxd->t_readable),
        (rd->t_generic, t_dte, t_sysbin),
        (auto->d_login, d_log);
```

- Subject in *d_daemon* can invoke logging process
- Can log, but not execute anything

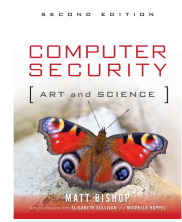
```
assign -r t_log /usr/var/log;
assign t_writable /usr/var/log/wtmp, /usr/var/log/utmp;
```

- Set type of logs



Low-Level Policy Languages

- Set of inputs or arguments to commands
  - Check or set constraints on system
- Low level of abstraction
  - Need details of system, commands

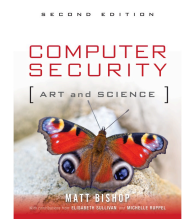


Example: X Window System

- UNIX X11 Windowing System
- Access to X11 display controlled by list
  - List says what hosts allowed, disallowed access

```
xhost +groucho -chico
```

- Connections from host groucho allowed
- Connections from host chico not allowed



Example: tripwire

- File scanner that reports changes to file system and file attributes
  - *tw.config* describes what may change

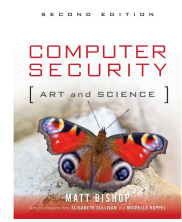
    `/usr/mab/tripwire +gimnpsu012345678-a`
    - Check everything but time of last access ("`-a`")
  - Database holds previous values of attributes



Example Database Record

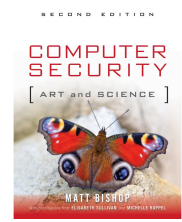
```
/usr/mab/tripwire/README 0 ..../. 100600 45763 1 917 10 33242
  .gtPvf .gtPvY .gtPvY 0 .ZD4cc0Wr8i21ZKaI..LUOr3
  .0fwo5:hf4e4.8TAqd0V4ubv ?..... ...9b3 1M4GX01xbGIX0oVuGo1h15z3
  ?:Y9jfa04rdzM1q:eqt1APgHk ?.Eb9yo.2zkEh1XKovX1:d0wF0kfAvC
  ?1M4GX01xbGIX2947jdyrior38h15z3 0
```

- file name, version, bitmask for attributes, mode, inode number, number of links, UID, GID, size, times of creation, last modification, last access, cryptographic checksums



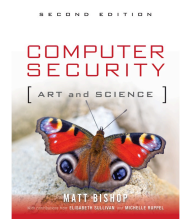
Comments

- System administrators not expected to edit database to set attributes properly
- Checking for changes with tripwire is easy
  - Just run once to create the database, run again to check
- Checking for conformance to policy is harder
  - Need to either edit database file, or (better) set system up to conform to policy, then run tripwire to construct database



Example English Policy

- Computer security policy for academic institution
  - Institution has multiple campuses, administered from central office
  - Each campus has its own administration, and unique aspects and needs
- Deals with electronic communications
  - Policy
  - User Advisories
  - Implementation at University of California Davis



Background

- University of California
  - 10 campuses (including UC Davis), each run by a Chancellor
  - UC Office of the President (UCOP) runs system, and is run by President of University of California
- UCOP issues policies that apply to all campuses
- Campuses implement the policy in a manner consistent with directions from UCOP



Electronic Communications Policy

- Begins with purpose, to whom policy applies
  - Includes email, video, voice, other means
  - Not to printed copies of communications
  - Not to Dept. of Energy labs that UC manages, or to Dept. of Energy employees
- Gives general implementation guidelines



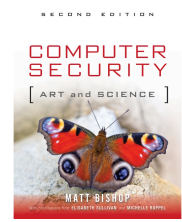
Use of Electronic Communications

- University does *not* want to deal with contents of these!
  - But all communications relating to University administration are public records
  - Others may be too
- Allowable users
  - Faculty, staff, students, others associated with UC
  - Others authorized by the Chancellors or UCOP
  - Others participating in programs UC sponsors



Allowable Uses

- University business
  - Classes, research, *etc.*
- Incidental personal use OK
  - But can't interfere with other uses
- Anonymous communications OK
  - But can't use a false identity



Non-Allowable Uses

- Endorsements not OK
- Running personal businesses not OJK
- Illegal activities not OK
  - Must respect intellectual property laws, US DMCA
- Violating University of campus policies or rules not OK
- Users can't put "excessive strain" on resources
  - No spamming, DoD or DDoS attacks



Privacy, Confidentiality

- General rule: respected the same way as is for paper
- Cannot read or disclose without permission of holder, except in specific circumstances
- To do so requires written permission of:
  - A designated Vice Chancellor (campus)
  - A Senior Vice President, Business and Finance (UCOP)



Privacy, Confidentiality

- Written permission not required for:
  - Subpoena or search warrant
  - Emergency
    - But must obtain approval as soon as possible afterwards
  - In all these cases, must notify those affected by the disclosure that the disclosure occurred, and why



Limits of Privacy

- Electronic communications that are public records will not be confidential
- Electronic communications may be on backups
- Electronic communications may be seen during routine system monitoring, etc.
  - Admins instructed to respect privacy, but *will* report "improper governmental activity"



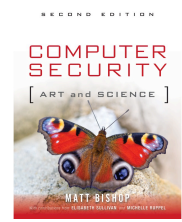
Security Services, Practices

- Routine monitoring
- Need for authentication
- Need for authorization
- Need for recovery mechanisms
- Need for audit mechanisms
- Other mechanisms to enforce University policy



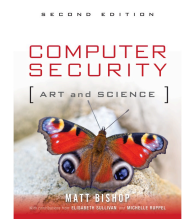
User Advisories

- These are less formal, give guidelines for the use of electronic communications
  - Show courtesy and consideration as in non-electronic communications
  - Laws about privacy in electronic communications are not as mature as laws about privacy in other areas
  - University provides neither encryption nor authentication
    - Easy to falsify sender



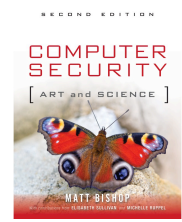
UC Davis Implementation

- Acceptable Use Policy
  - Incorporates the UCD Principles of Community
  - Requires respect of rights of others when using electronic communications
  - Use encouraged for education, university business, university-related activities



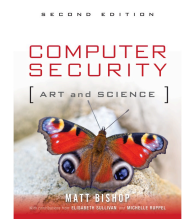
UC Davis Implementation

- UC Davis specific details
  - Only Chancellor-approved charitable activities may use these resources
  - Cannot be used to create hostile environment
    - This includes violating obscenity laws
  - Incidental personal use OK under conditions given in Electronic Communications Policy



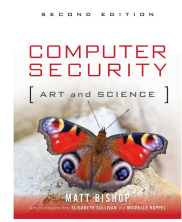
UC Davis Implementation

- Unacceptable conduct
  - Not protecting passwords for University resources
  - Not respecting copyrights, licenses
  - Violating integrity of these resources
  - Creating malicious logic (worms, viruses, *etc*.)
    - Allowed if done as part of an academic research or instruction program supervised by academic personnel; and
    - It does not compromise the University's electric communication resource



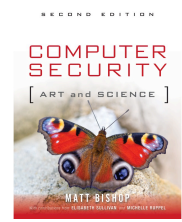
UC Davis Implementation

- Allowed users
  - UCD students, staff, faculty
  - Other UCD academic appointees and affiliated people
    - Such as postdocs and visiting scholars
- People leaving
  - Forwarding email allowed
  - Recipient must agree to return to the University any email about University business



Exceptions Allowing Disclosure

- Required by law;
- Reliable evidence of violation of law, University policies;
- Failure to do so may result in:
  - Significant harm
  - Loss of significant evidence of violations;
  - Significant liability to UC or its community;
- Not doing so hampers University meeting administrative, teaching obligations



Secure, Precise Mechanisms

- Can one devise a procedure for developing a mechanism that is both secure *and* precise?
  - Consider confidentiality policies only here
  - Integrity policies produce same result
- Program a function with multiple inputs and one output
  - Let p be a function p: $I_1 \times \ldots \times I_n \rightarrow R$. Then p is a program with n inputs $i_k \in I_k$, $1 \leq k \leq n$, and one output $r \rightarrow R$



Programs and Postulates

- Observability Postulate: the output of a function encodes all available information about its inputs
  - Covert channels considered part of the output
- Example: authentication function
  - Inputs name, password; output Good or Bad
  - If name invalid, immediately print Bad; else access database
  - Problem: time output of Bad, can determine if name valid
  - This means timing is part of output

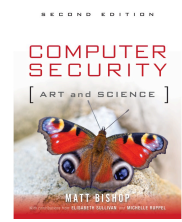


Protection Mechanism

- Let p be a function p: $I_1 \times \ldots \times I_n \rightarrow R$. A *protection mechanism* m is a function

$$m: I_1 \times \ldots \times I_n \rightarrow R \cup E$$

  for which, when $i_k \in I_k$, $1 \le k \le n$, either
  - $m(i_1, \ldots, i_n) = p(i_1, \ldots, i_n)$ or
  - $m(i_1, \ldots, i_n) \in E$.
- E is set of error outputs
  - In above example, E = { “Password Database Missing”, “Password Database Locked” }



Confidentiality Policy

- Confidentiality policy for program p says which inputs can be revealed
  - Formally, for p: $I_1 \times \ldots \times I_n \rightarrow R$, it is a function c: $I_1 \times \ldots \times I_n \rightarrow A$, where
    $$A \subseteq I_1 \times \ldots \times I_n$$
  - A is set of inputs available to observer
- Security mechanism is function
    $$m\text{: } I_1 \times \ldots \times I_n \rightarrow R \cup E$$
  - m is *secure* if and only if $\exists\, m'\text{: } A \rightarrow R \cup E$ such that,
    $$\forall i_k \in I_k,\ 1 \le k \le n,\ m(i_1, \ldots, i_n) = m'(c(i_1, \ldots, i_n))$$
  - m returns values consistent with c



Examples

- $c(i_1, ..., i_n) = C$, a constant
  - Deny observer any information (output does not vary with inputs)
- $c(i_1, ..., i_n) = (i_1, ..., i_n)$, and $m' = m$
  - Allow observer full access to information
- $c(i_1, ..., i_n) = i_1$
  - Allow observer information about first input but no information about other inputs.



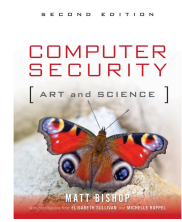
Precision

- Security policy may be over-restrictive
  - Precision measures how over-restrictive
- m_1, m_2 distinct protection mechanisms for program p under policy c
  - *m_1 as precise as m_2* ($m_1 \approx m_2$) if, for all inputs i_1, …, i_n, $m_2(i_1, …, i_n) = p(i_1, …, i_n) \Rightarrow m_1(i_1, …, i_n) = p(i_1, …, i_n)$
  - *m_1 more precise than m_2* ($m_1 \sim m_2$) if there is an input $(i_1', …, i_n')$ such that $m_1(i_1', …, i_n') = p(i_1', …, i_n')$ and $m_2(i_1', …, i_n') \neq p(i_1', …, i_n')$.



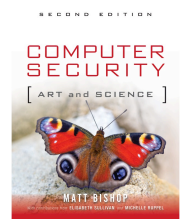
Combining Mechanisms

- m_1, m_2 protection mechanisms
- $m_3 = m_1 \cup m_2$
  - For inputs on which m_1 and m_2 return same value as p, m_3 does also; otherwise, m_3 returns same value as m_1
- Theorem: if m_1, m_2 secure, then m_3 secure
  - Also, $m_3 \approx m_1$ and $m_3 \approx m_2$
  - Follows from definitions of secure, precise, and m_3



Existence Theorem

- For any program *p* and security policy *c*, there exists a precise, secure mechanism *m** such that, for all secure mechanisms *m* associated with *p* and *c*, *m** ≈ *m*
  - Maximally precise mechanism
  - Ensures security
  - Minimizes number of denials of legitimate actions



Lack of Effective Procedure

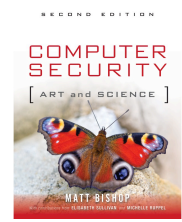
- There is no effective procedure that determines a maximally precise, secure mechanism for any policy and program.
  - Sketch of proof: let policy *c* be constant function, and *p* compute function *T*(*x*). Assume *T*(*x*) = 0. Consider program *q*, where

```
p;
if z = 0 then y := 1 else y := 2;
halt;
```



Rest of Sketch

- *m* associated with *q*, *y* value of *m*, *z* output of *p* corresponding to $T(x)$
- $\forall x[T(x) = 0] \rightarrow m(x) = 1$
- $\exists x' [T(x') \neq 0] \rightarrow m(x) = 2$ or $m(x)$ undefined
- If you can determine *m*, you can determine whether $T(x) = 0$ for all *x*
- Determines some information about input (is it 0?)
- Contradicts constancy of *c*.
- Therefore no such procedure exists



Key Points

- Policies describe *what* is allowed
- Mechanisms control *how* policies are enforced
- Trust underlies everything